

Constraint-Based Problem Solving

Program Execution Output — Questions 1, 2 and 3

Question 1: Backtracking Search — Execution Trace

Console output of the backtracking CSP solver, showing each trial assignment, pruning/backtrack events, and the final valid schedule found.

1)

```
Run ass x
Step 1: Trying D1 = Morning (1)
-> Valid partial assignment: {'D1': 1}
Step 2: Trying D2 = Morning (1)
-> [PRUNED / BACKTRACK] Constraint violation at {'D1': 1, 'D2': 1}
Step 3: Trying D2 = Afternoon (2)
-> Valid partial assignment: {'D1': 1, 'D2': 2}
Step 4: Trying D3 = Morning (1)
-> [PRUNED / BACKTRACK] Constraint violation at {'D1': 1, 'D2': 2, 'D3': 1}
Step 5: Trying D3 = Afternoon (2)
-> [PRUNED / BACKTRACK] Constraint violation at {'D1': 1, 'D2': 2, 'D3': 2}
Step 6: Trying D3 = Night (3)
-> Valid partial assignment: {'D1': 1, 'D2': 2, 'D3': 3}

[GOAL REACHED] Valid Assignment Found: {'D1': 1, 'D2': 2, 'D3': 3}

=====
      FINAL VALID SCHEDULE
=====
D1 -> Morning
D2 -> Afternoon
D3 -> Night
```

2)

```
Run ass x
Step 14:
  Popped Node: (2, 3) | g = 5, h = 3, f = 8
  Expanded Neighbor (2, 4): g=6, h=2, f=8

Step 15:
  Popped Node: (1, 4) | g = 5, h = 3, f = 8

Step 16:
  Popped Node: (4, 2) | g = 6, h = 2, f = 8
  Expanded Neighbor (4, 3): g=7, h=1, f=8

Step 17:
  Popped Node: (2, 4) | g = 6, h = 2, f = 8
  Expanded Neighbor (3, 4): g=7, h=1, f=8

Step 18:
  Popped Node: (4, 3) | g = 7, h = 1, f = 8
  Expanded Neighbor (4, 4): g=8, h=0, f=8

Step 19:
  Popped Node: (3, 4) | g = 7, h = 1, f = 8

Step 20:
  Popped Node: (4, 4) | g = 8, h = 0, f = 8

>>> GOAL REACHED! <<<

=====
Optimal Path: [(0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (4, 1), (4, 2), (4, 3), (4, 4)]
Total Path Cost: 8
=====
```

Question 2: Search Trace — Execution Output

Console output of the grid search program, showing popped nodes with their g, h and f values, expanded neighbours, and the goal-reached confirmation with the optimal path and total path cost.

```
Run ass x
Step 7:
  Popped Node: (0, 3) | g = 3, h = 5, f = 8
  Expanded Neighbor (1, 3): g=4, h=4, f=8
  Expanded Neighbor (0, 4): g=4, h=4, f=8

Step 8:
  Popped Node: (4, 0) | g = 4, h = 4, f = 8
  Expanded Neighbor (4, 1): g=5, h=3, f=8

Step 9:
  Popped Node: (3, 1) | g = 4, h = 4, f = 8
  Expanded Neighbor (3, 2): g=5, h=3, f=8

Step 10:
  Popped Node: (1, 3) | g = 4, h = 4, f = 8
  Expanded Neighbor (2, 3): g=5, h=3, f=8
  Expanded Neighbor (1, 4): g=5, h=3, f=8

Step 11:
  Popped Node: (0, 4) | g = 4, h = 4, f = 8

Step 12:
  Popped Node: (4, 1) | g = 5, h = 3, f = 8
  Expanded Neighbor (4, 2): g=6, h=2, f=8

Step 13:
  Popped Node: (3, 2) | g = 5, h = 3, f = 8
  Expanded Neighbor (2, 2): g=6, h=4, f=10
```

```
Run ass x
C:\Users\teeka\PycharmProjects\PythonProject\venv\Scripts\python.exe C:\Users\teeka\PycharmProjects\PythonProject\ass.py
=====
      A* SEARCH STEP-BY-STEP TRACE
=====

Step 1:
  Popped Node: (0, 0) | g = 0, h = 8, f = 8
  Expanded Neighbor (1, 0): g=1, h=7, f=8
  Expanded Neighbor (0, 1): g=1, h=7, f=8

Step 2:
  Popped Node: (1, 0) | g = 1, h = 7, f = 8
  Expanded Neighbor (2, 0): g=2, h=6, f=8

Step 3:
  Popped Node: (0, 1) | g = 1, h = 7, f = 8
  Expanded Neighbor (0, 2): g=2, h=6, f=8

Step 4:
  Popped Node: (2, 0) | g = 2, h = 6, f = 8
  Expanded Neighbor (3, 0): g=3, h=5, f=8

Step 5:
  Popped Node: (0, 2) | g = 2, h = 6, f = 8
  Expanded Neighbor (0, 3): g=3, h=5, f=8

Step 6:
  Popped Node: (3, 0) | g = 3, h = 5, f = 8
  Expanded Neighbor (4, 0): g=4, h=4, f=8
  Expanded Neighbor (3, 1): g=4, h=4, f=8
```

Question 3: Online Rescue Robot Simulation — Execution Output

Console output of the online rescue-robot navigation simulation, showing the robot moving step-by-step toward the survivor, sensing each cell, accumulating path cost, and finally locating the survivor with the optimal path and total path cost reported.

3)

```
Run ass x
C:\Users\teeka\PycharmProjects\PythonProject\.venv\Scripts\python.exe C:\Users\teeka\PycharmProjects\PythonProject\ass.py

=====
ONLINE RESCUE ROBOT NAVIGATION SIMULATION
=====
Step 1: Moved to (1, 0) | Cell: Clear (Cost=1) | Path Cost: 1
Step 2: Moved to (2, 0) | Cell: Clear (Cost=1) | Path Cost: 2
Step 3: Moved to (3, 0) | Cell: Clear (Cost=1) | Path Cost: 3
Step 4: Moved to (4, 0) | Cell: Clear (Cost=1) | Path Cost: 4
Step 5: Moved to (4, 1) | Cell: Clear (Cost=1) | Path Cost: 5
Step 6: Moved to (4, 2) | Cell: Clear (Cost=1) | Path Cost: 6
Step 7: Moved to (4, 3) | Cell: Clear (Cost=1) | Path Cost: 7
Step 8: Moved to (4, 4) | Cell: Clear (Cost=1) | Path Cost: 8

=====
>>> SURVIVOR LOCATED SUCCESSFULLY! <<<
Optimal Path Taken: [(0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (4, 1), (4, 2), (4, 3), (4, 4)]
Total Path Cost: 8
=====

This view is read-only
Press Ctrl+Q to quit with exit code 0
```